CSE 3102L: WEB TECHNOLOGY LAB

Course Teachers:

Dr. Nurul Ahad Tawhid
Associate Professor, IIT University of Dhaka
Mridha Md. Nafis Fuad
Lecturer, IIT, University of Dhaka

Institute of Information Technology
University of Dhaka

1. Course Overview

This lab course introduces the concepts, tools and working practices used to develop and maintain modern web applications. The course is designed to help students move beyond isolated programming exercises and gain practical experience with tasks commonly assigned to entry-level web developers during an industry internship.

The course is technology-agnostic. Students may use any suitable programming language, framework, database, development tool or AI coding assistant. The emphasis is on understanding concepts that apply across technologies, selecting appropriate tools, implementing features, testing results and explaining technical decisions.

Course objectives

- Understand how browsers, clients, servers and web services communicate.
- Develop and consume APIs and integrate frontend and backend components.
- Implement authentication, authorization and secure data handling.
- Work with databases, files, streams, caches and background processing.
- Apply software testing, debugging, security and performance practices.
- Use version control, code review, automated build/test workflows and deployment practices.
- Read and extend an existing codebase, investigate issues and deliver maintainable changes.
- Use AI development assistants responsibly while retaining ownership of submitted work.

Expected learning outcomes

By the end of the course, students should be able to:

- Explain the HTTP request-response lifecycle and inspect web traffic.
- Design, implement, document and test REST-style APIs.
- Build a web application using a technology stack of their choice.
- Implement password-based authentication and role/permission-based authorization, and explain OAuth 2.0, OpenID Connect, and passkeys.
- Persist and retrieve structured data and manage uploaded files.
- Use caching and asynchronous job processing for suitable application tasks.
- Write and run automated tests, debug defects and address common web security risks.
- Collaborate through Git branches, commits, pull requests and code reviews.
- Configure a basic CI workflow and deploy or reproducibly run an application.

2. Class Format

Each three-hour session will generally follow this structure. The exact allocation may be adjusted depending on the activity.

Session segment	Time	What to expect
Concept introduction and demonstration	30 minutes	Core concepts, architecture, examples and tool demonstrations.
Guided exercise	30 minutes	A short instructor-guided task to establish the workflow.
Hands-on lab work	90 minutes	Individual or group implementation, investigation, testing and debugging.
Review and submission	30 minutes	Demonstration, discussion, feedback and submission.

3. Course Plan

The sessions are organized around a progressively developed application. The application scenario and requirements will be provided during class. Students should maintain their work in a version-controlled repository and extend it throughout the course.

Topic 01 — Web Fundamentals, HTTP and Developer Workflow

Objective: Understand the web request-response model and become familiar with essential development and debugging tools.

Topics: Client-server architecture; DNS; HTTP and HTTPS; request and response structure.; HTTP methods, headers, status codes, cookies, query parameters and request bodies.; Browser DevTools, network inspection, and command-line/API clients.

Practical work:

- Inspect requests and responses using browser DevTools and an API client such as Postman or an equivalent tool.
- Run a mock API (for example, JSON Server) and test GET, POST, PUT/PATCH and DELETE.
- Record experiments involving headers, status codes, invalid requests and error responses.
- Create a repository and make the first commits.

Expected submission: API request collection and a concise HTTP experiment record.

Topic 02 — JavaScript for Web Development

Objective: Use JavaScript to implement browser interactions and communicate with web services.

Topics: Modern JavaScript syntax; variables, functions, objects and arrays; modules and JSON.; DOM manipulation, events, forms and client-side validation.; Callbacks, Promises, async/await, event-loop basics and Fetch API.

Practical work:

- Build an interactive page that displays and filters data retrieved from an API.
- Implement create, update and delete operations using asynchronous requests.
- Handle loading, empty, success and error states.
- Compare Promise chaining with async/await.

Expected submission: A working JavaScript client that consumes an API and handles common response states.

Topic 03 — REST APIs and Web Services

Objective: Design APIs around resources and implement predictable, documented interfaces.

Topics: REST principles, resources, URI design, CRUD mapping and statelessness.; Request validation, response structure, status-code conventions and error formats.; API contracts, pagination, filtering, sorting and versioning.

Practical work:

- Design endpoints for the selected application scenario.
- Write a brief API contract before implementation.
- Implement or configure the API and test normal, invalid and boundary cases.
- Use environment variables and reusable request collections in the API client.

Expected submission: API specification and a tested set of API endpoints.

Topic 04 — Backend Development and Architecture

Objective: Build server-side functionality and organize code so that it can be extended and maintained.

Topics: Server-side request handling, routing and middleware.; Separation of concerns; controllers/handlers, services, repositories and domain logic.; Dependency management, configuration, centralized error handling and logging.

Practical work:

- Implement backend endpoints using a framework or platform of choice.
- Refactor a small or supplied codebase into understandable modules/layers.
- Add validation, logging and centralized error handling.
- Connect the backend to the existing API client or frontend.

Expected submission: Backend implementation with a clear structure, validation and error handling.

Topic 05 — Authentication and Authorization

Objective: Implement secure user access and distinguish identity verification from permission enforcement.

Topics: Registration, login and logout; password hashing and verification.; Sessions, cookies, JWTs, access tokens and refresh-token concepts.; Authentication vs authorization; roles, permissions and access control.; OAuth 2.0, OpenID Connect, passkeys, third-party sign-in and single sign-on concepts.; Common risks: plaintext passwords, token leakage, weak session handling, CSRF and insecure access checks.

Practical work:

- Implement registration and password-based login using a suitable, established library.
- Store password hashes, not plaintext passwords.
- Protect selected API routes and enforce role/permission rules.
- Test unauthorized access, insufficient permissions, logout and expiry.
- Where feasible, integrate a trusted identity provider for third-party sign-in.

Expected submission: Authenticated application workflows and evidence of authorization tests. OAuth/OIDC should be integrated using established libraries/providers; students are not expected to implement the protocols from scratch.

Topic 06 — Database and Data Management

Objective: Persist application data and maintain relationships and consistency.

Topics: Relational and NoSQL database concepts and selection considerations.; Tables/collections, keys, relationships, constraints and indexes.; Basic SQL and data access through drivers or ORMs.; Migrations, transactions, consistency and connection management.

Practical work:

- Design and implement a database schema for the application.
- Implement data access and CRUD operations.
- Add relationships, constraints and appropriate indexes.
- Use migrations and test transaction or consistency requirements.

Expected submission: Database schema, migration files and working persistent data operations.

Topic 07 — File Management and Data Streaming

Objective: Handle user-provided files and large data efficiently and safely.

Topics: Upload and download workflows; multipart requests and file metadata.; File type/size validation, naming, storage location and access permissions.; Local storage vs object/cloud storage concepts.; Streaming, buffering, chunking and handling large files without loading everything into memory.

Practical work:

- Implement file upload and download for an application feature.
- Validate file size and allowed formats; prevent unsafe file access.
- Store metadata and associate files with application records.
- Demonstrate streaming or chunked handling for a larger file where supported.

Expected submission: A working file-management feature with validation and documented storage behavior.

Topic 08 — Caching and Performance

Objective: Recognize performance bottlenecks and use caching appropriately.

Topics: Browser, HTTP, server-side and distributed caching concepts.; Cache-Control, ETags and conditional requests.; Cache keys, expiration, invalidation and stale data.; CDNs, database query optimization and connection pooling concepts.

Practical work:

- Identify a frequently requested or expensive operation.
- Add an appropriate cache at one layer of the application.
- Compare behavior or response time before and after caching.

- Demonstrate expiration or invalidation and investigate stale-data behavior.

Expected submission: A documented caching implementation and a before/after observation.

Topic 09 — Queues and Background Processing

Objective: Move long-running or non-immediate work out of user-facing request handling.

Topics: Synchronous vs asynchronous processing; producers, consumers and workers.; Message queues, job queues and scheduled/delayed jobs.; Retries, backoff, idempotency, failure handling and dead-letter queues.; When to use background processing and its operational trade-offs.

Practical work:

- Implement a background task such as email notification, report generation or media processing.
- Queue a job from an application request and process it with a worker.
- Add a delayed or scheduled job.
- Demonstrate retry/failure behavior and prevent duplicate processing where appropriate.

Expected submission: A working background-processing workflow with evidence of job status and failure handling.

Topic 10 — Frontend Engineering and Full-Stack Integration

Objective: Build a modern, responsive frontend and integrate it with backend services using appropriate rendering architectures and web application technologies.

Topics: Component-based UI development, routing, forms and state management; SPA, MPA, CSR, SSR and SSG concepts and trade-offs; frontend/backend boundaries, API clients and data flow; authentication state, protected views and role-based interfaces; PWA fundamentals, service workers, caching and offline support; responsive design, accessibility and frontend performance.

Practical work:

- Apply appropriate rendering approaches (SPA/CSR, SSR or SSG).
- Integrate authentication, session management and role-specific views.
- Implement PWA features, including installability, caching and basic offline support.
- Handle loading, validation, network and authorization errors.
- Ensure responsive design, accessibility and performance.

Expected submission: An integrated, responsive frontend and backend application implementing the principal workflows, with appropriate rendering architecture, authentication and essential PWA capabilities.

Topic 11 (if time permits) — Testing, Debugging and Web Security

Objective: Verify behavior systematically, diagnose defects and address common security weaknesses.

Topics: Unit, integration, API and end-to-end testing; test pyramid and TDD.; Assertions, fixtures, mocks and test data.; Debugging, logs, reproduction steps and root-cause analysis.; Common web risks: injection, XSS, CSRF, broken access control, insecure configuration and sensitive-data exposure.

Practical work:

- Write tests for important business rules and API endpoints.
- Reproduce and fix an intentionally introduced defect.
- Test authentication and authorization boundaries.
- Review the application for common security issues and document findings.

Expected submission: Automated tests, a fixed defect and a short test/security summary.

Topic 12 (if time permits) — Collaboration, CI/CD and Deployment

Objective: Practice the collaboration and delivery workflow used by software teams.

Topics: Git branches, commits, merge conflicts, pull requests and code review.; Environment configuration and secret management.; Build/test automation, CI/CD concepts and deployment environments.; Containers, health checks, logs and basic monitoring.

Practical work:

- Implement a small feature on a separate branch.
- Submit or simulate a pull request and review another implementation.
- Configure an automated build/test workflow.
- Deploy the application or provide a reproducible local/containerized deployment.

Expected submission: Feature branch/PR evidence, automated workflow and deployment or reproducible run instructions.

Topic 13 (if time permits) — Integration and Production Readiness

Objective: Validate the complete system and communicate implementation decisions clearly.

Topics: End-to-end integration, reliability and graceful failure.; Performance checks, observability, maintainability and technical documentation.; Architecture explanation, known limitations and demonstration practices.

Practical work:

- Run complete user workflows and verify database, file, authentication and background-job behavior.
- Investigate and fix integration defects.
- Prepare setup instructions, architecture notes and API documentation.
- Demonstrate the application and explain key technical decisions.

Expected submission: Final lab demonstration, repository and concise technical documentation.

4. Technology and Tool Policy

No single framework or technology stack is mandatory for the practical work unless a particular exercise requires a shared tool for demonstration or assessment. Students may select technologies appropriate to their project and experience.

- Students are responsible for selecting, installing and configuring the tools they use.
- The implementation must demonstrate the required concepts regardless of the selected framework.
- Students should be able to explain why a technology or library was selected and identify relevant alternatives.
- Use official documentation and maintain a reproducible setup guide.
- Do not commit passwords, API keys, tokens, private keys or other secrets to a repository.
- Where a paid service or external account is required, an equivalent free or local alternative may be used with instructor approval.

5. Continuous Assessment

Continuous assessment carries 70 marks. The final project will be assessed separately.

Component	Marks	Assessment basis
Lab exercises and submissions	50	Practical implementation, required concepts, explanation and completeness.
Code quality and development practices	10	Readability, modularity, version control, validation/error handling and documentation.
Participation and engagement	10	Practical involvement, investigation, technical discussion and collaboration.
TOTAL	**70**	

6. Submission and Make-up Policy

Unless a different deadline is announced, the following late-submission scale applies to lab submission marks:

Submission time	Maximum marks awarded
By the deadline	100%
1–2 calendar days late	90%
3–4 calendar days late	80%
5–7 calendar days late	70%
More than 7 days late	Normally not accepted without instructor approval

Approved extensions will follow the revised deadline and will not incur a late penalty when the work is submitted within that extension.

Absence and make-up work

- Lab performace and lab submission are assessed separately. Completing a make-up task does not change the attendance record for the missed session.
- A student who attends but does not complete the lab will receive submission marks based on the work actually demonstrated or submitted.
- For an approved absence, the instructor may waive the late penalty and set an appropriate extension.

Technical difficulties

Report problems such as device failure, internet outage, account/service unavailability or corrupted files as soon as possible. Keep reasonable evidence where available. The instructor may approve an extension or an alternative method of demonstrating the same learning outcomes. Students are expected to maintain backups and commit work regularly.

7. AI Tool and Academic Integrity Policy

AI coding assistants and other AI tools may be used in this course. Using AI is not a substitute for understanding, testing or taking responsibility for submitted work.

- Students may use AI tools to understand concepts, explore alternatives, generate code, write tests, debug errors and improve documentation.
- Students must review and verify generated code, including checking library/API usage against reliable documentation.
- Students are responsible for the correctness, security, licensing and maintainability of everything they submit.
- Do not submit code that you cannot explain or modify when asked.
- Do not share passwords, access tokens, API keys, private credentials, personal records or confidential project data with AI tools.
- Disclose substantial AI assistance briefly in the submission. State what the tool helped with and what you verified or changed.
- Copying another student's work or presenting another person's work as your own is not acceptable. Give attribution where required.
- Similarity alone may not establish misconduct; however, inability to explain or reproduce submitted work may be considered during assessment.
-

The goal is not to master one framework. The goal is to become capable of learning, building, testing and maintaining web applications in a professional development environment.